Gaps in Operator Fusion for Deep Learning Systems: The *Trade-off* Between Compilation and Execution of Unfused Operations

Anonymous Author(s)

ABSTRACT

Operator fusion trades compilation time for execution efficiency, but this cost is often hidden. In modern compilers such as PyTorch 2 (*TorchInductor*) and XLA, heuristics and constraint rules prevent the fusion of common kernels like BatchNorm+ReLU and BatchNorm → Conv, which, if fused, can affect both compilation and execution times. In this paper, we algebraically formalize the trade-off between compilation-time and execution-time gains during Operator Fusion passes in modern machine learning compilers. At the core of our proposal is a framework that models the total cost of a backend as $T(n) = T_C + n\,T_X$, where T_C is the compilation time, T_X the per-execution latency, and n the number of executions, making explicit the regime in which each compiler is preferable. We also show empirically that, with respect to time-to-first-batch, PyTorch 2 performs fusion passes in a suboptimal order, and we propose an alternative optimization sequence. Finally, we provide a real-world benchmark and a tool to indicate the best backend according to the usage regime.

KEYWORDS

Operator Fusion, Machine Learning Compilers, Compilation Time, Kernel Fusion, Performance Benchmarking

1 Introduction

The *operator fusion* technique has direct roots in classical compiler optimizations, such as *loop* fusion, historically used to increase parallelism and improve data locality [13]. In modern compilers for Machine Learning, particularly on GPUs, this technique plays a central role: by combining multiple operations into a single *kernel*, memory traffic is reduced, intermediate tensors are avoided, and the number of dynamic launches during execution is decreased [4, 9]. Systems such as XLA, TVM, TensorRT, and the TorchInductor backend adopt aggressive fusion to amortize CPU–GPU communication costs and consolidate operator sequences into larger, more efficient *kernels* [1, 4, 8].

Fusion is performed over the program's control-flow graph (CFG). However, deciding *what* and *how* to fuse in modern graphs is challenging: optimal fusion plans are, in general, NP-hard problems [5, 7]. For this reason, compilers adopt heuristics that seek to balance **compilation time** and **execution time**. This balance is especially important in dynamic workloads (with variable *shapes*) and in scenarios with few repetitions, where the cost up to the first execution dominates the perceived performance.

Despite this impact, the literature usually favors latency metrics and ignores compilation time. There is no standard protocol for making explicit when it pays off to compile more aggressively in exchange for faster execution, leaving practitioners without a principled way to choose among backends. XLA [12], TVM [4], and TorchInductor (PyTorch 2) [1] each take a different approach—rule-based fusion, cost-model-guided exploration, and local code generation, respectively—resulting in very different compilation- and execution-time profiles across usage regimes.

We evaluate the *trade-off* between these two times across the TVM, XLA, and TorchInductor compilers under a common protocol. The core of our proposal is a framework that, for a number of executions n, models the total cost of each *backend* as $T_b(n) = a_b n + b_b = T_C + n\,T_X$, where $b_b = T_C$ is the compilation time and $a_b = T_X$ is the per-execution latency (Section 3); minimizing $T_b(n)$ over the backends identifies which one is preferable in each usage regime. We also investigate the fusion heuristics used by the compilers and propose a new execution order that simultaneously reduces compilation time and execution time on real-world models.

Contributions:

- An algebraic framework, $T(n) = T_C + n\,T_X$, formalizing the trade-off between compilation time (T_C) and execution time (T_X) of ML (*Machine Learning*) models with respect to *kernel* fusion heuristics;
- Analysis and benchmarking of the XLA, PyTorch, and TVM models in terms of our algebraic framework;
- Artifact analysis (TVMScript, HLO, Triton) to quantify and analyze fusion granularity per *backend*, mapping patterns by architecture (ResNet-18/50, BERT, and GPT-2) and their impact on latency;

2 Theoretical Background

Consider a sequence of operators $x_{k+1} = f_k(x_k)$ and $x_{k+2} = f_{k+1}(x_{k+1})$, in which each f_i acts on tensors of a computation graph. We say that f_k and f_{k+1} are *fusible* when there exists a single operator h such that $h(x_k) = f_{k+1}(f_k(x_k))$ and the execution of h preserves every observable semantics of the original sequence. In this case, the compiler performs the transformation $(f_k, f_{k+1}) \Rightarrow h$, eliminating the materialization of intermediate tensors, reducing the number of kernel launches, and improving memory locality.

This notion of fusibility is the computation-graph analogue of classical *peephole optimization*, as we detail in Section 5: h plays, for a window of graph operators, the role that an equivalent instruction plays for a window of three-address-code instructions. In ML models, however, operators handle multi-dimensional tensors and their definitions are often complex. We therefore focus on three fusions, defined by Equations (III), (VIII), and (XIV) in Appendix A: `BatchNorm+ReLU`, `Conv+BatchNorm+ReLU`, and the affine part of `LayerNorm` folded into `Linear`.

Some kernel sequences are intentionally left unfused, owing to conservative rules that prevent incorrect execution or unpredictable performance:

- **Training mode.** In training, `BatchNorm` depends on the statistics of the current batch and on the update of internal state. Fusing it may break correctness, since it prevents the reuse of intermediate values needed for the gradient.
- **Canonical form.** Fusion requires the operators to be in direct sequence in the CFG, with no intervening operations and no multiple consumers, which preserves the original semantics.
- **Numerical precision.** Precision mismatches (e.g., `BatchNorm` in FP32 and `ReLU` in FP16) can make fusion unsafe due to overflow or underflow risks.
- **Shape compatibility.** Fusion candidates must have compatible shapes; mismatches may require reindexing or broadcasting, making fusion infeasible.
- **Operation type.** Not every operation is fusible: differing memory-access patterns or synchronization requirements may prevent execution in a single kernel.
- **Backend restrictions.** Calls to external libraries (e.g., cuDNN or cuBLAS) and non-deterministic operations limit fusibility, since the compiler does not control their internal execution.

In every case, the fusion decision involves an explicit trade-off: investing more compilation time to reduce execution time, or compiling quickly with fewer fusions. This balance underlies the cost model adopted in this work and guides the comparison between backends.

3 Methodology

We compare *TVM* [4], *XLA* [12], and *TorchInductor* (PyTorch 2) [1] in a controlled environment, measuring **compilation time**, **average latency per execution**, and **fusion granularity**.

3.1 Experimental Environment

All experiments were run on the same computing system. The base environment consists of a Fedora 41 operating system, using kernel 6.16.12, with an AMD Ryzen 5 4600G CPU and 15.4 GiB of RAM. The GPU was an NVIDIA RTX 3050 with 8 GB of VRAM, operating with driver version 580.95.05. The CUDA stack comprised the CUDA 13.0 driver and the *nvcc* compiler version 12.6 (V12.6.77).

Per-framework configuration. For the **TVM** experiments, we used Python 3.11.13, cuDNN version 9.1 (via PyTorch), TVM 0.22.dev0 (commit 3b60f1c), LLVM 20.1.8, and CUDA 12.6 integrated into the TVM backend.

For **XLA/TorchInductor**, the experiments were conducted with Python 3.10.18. The XLA backend used JAX/jaxlib 0.6.2, with PJRT (`jax-cuda12-pjrt` 0.6.2) and cuDNN 9.10.2.21 installed via *pip*. For TorchInductor, we used PyTorch 2.5.1+cu121, executing on device `cuda:0`.

We used the same hardware/OS and software *stack* per *backend*. Standardized conditions: `TF32` enabled, *channels_last* layout, cud `nn.benchmark=True`, all models in `eval()`, *warmup*, and explicit separation between compilation and execution. We used an isolated environment for **TVM** version `0.22.dev0`.

3.2 Models and Inputs

We evaluated **ResNet-18/50**, **BERT**, and **GPT-2**. Inputs for ResNets (CNNs): `(1,3,224,224)`, `(16,3,224,224)`, `(64,3,224,224)`, `(1,3,512,512)`, `(1,3,1024,1024)`. Inputs for Transformers: `(batch, seq_len)` pairs in `(1,64)`, `(1,128)`, `(8,128)`.

The **ResNet-18/50**, **BERT**, and **GPT-2** models were chosen because they represent distinct and complementary architectural patterns. The ResNets characterize classical convolutional architectures, with recurring *Conv–BatchNorm–ReLU* chains, which are canonical cases for analyzing operator fusion in CNNs.

BERT represents *Transformer* architectures based on encoders, dominated by matrix multiplications, linear projections, normalizations, and activations, allowing us to evaluate fusion strategies on attention-centric graphs.

GPT-2, in turn, exemplifies autoregressive *decoder-only* models, with a more linear and repetitive computational flow, typical of generative models, in which fusion granularity directly affects inference latency.

Together, these models allow us to analyze the behavior of fusion heuristics across different computational and architectural regimes.

3.3 Experimental Pipeline

The benchmark workflow adopts four sequential stages, standardized across all models and backends (TVM, XLA, and TorchInductor):

(1) Compilation. For each *(model, input)* pair, the compilation time (`compile_ms`) is recorded in an isolated process with fixed driver and library versions, focusing primarily on the cost of code generation.

(2) Warmup and execution + graph capture. Execution occurs in two phases: (i) a *warmup* phase, focused on building caches and performing *autotuning*; and (ii) a phase of *valid*, timed *executions*. During (ii), the benchmark exports the CFGs of the intermediate representations (IRs) of the analyzed backends [HLO (XLA), TVM-Script and TIR (TVM), IR and Triton (TorchInductor)]. We also collect *kernel* metadata when available.

(3) Analysis of kernel quantity and quality. From the captured IRs, we perform a structural analysis of the *kernels* generated by each backend. This stage includes quantifying the total number of *kernels*, distinguishing between internal *kernels* and external library calls, as well as qualitatively characterizing the fusions performed. This analysis allows us to evaluate fusion granularity and the degree of delegation to optimized libraries, providing structural context for interpreting the performance results.

(4) Measurements and algebraic model. The metrics collected include: *time to first batch* (`ttfb_ms`), *average latency per execution a*, and *standard deviation* over the 50 valid executions (`exec_ms_mean`, `exec_ms_std`). For **XLA/TVM**, the compilation time is **explicitly** separated from the first compiled call *b*. For **TorchInductor**, the compilation time is estimated by

$$\text{compile\_time} = \text{first\_call\_ms} - \text{time\_per\_run\_ms},$$

where *first_call_ms* aggregates compilation+execution and *time_per_run_ms* is the average of the subsequent executions.

Algebraic model. We propose an algebraic model for computing the cost of compilation and batched execution across the three analyzed backends, given by:

$$T_b(n) = a_b\, n + b_b,$$

where b_b is the fixed cost of compilation and initial overhead, and a_b is the average latency, allowing us to infer the inflection point for different backends as a function of the number of repetitions n. Equivalently, writing $b_b = T_C$ (compilation time) and $a_b = T_X$ (per-execution latency), the model reads $T_b(n) = T_C + n\,T_X$; this is the framework we use throughout to compare backends.

The model $T_b(n) = a_b n + b_b$, together with its *lower envelope*—the curve that, for each value of n, selects the smallest cost among all backends—helps explain the crossover points observed between the different *backends*. Thus, the intersections of these functions indicate from which input size n each backend becomes more advantageous.

(5) Applying fold and comparing regimes. In addition to the standard models, we evaluated an alternative scenario in which a *fold* transformation is applied **before** the regular operator fusion. The *fold* consists of an algebraic reparameterization that absorbs linear operations (such as *BatchNorm* or the affine part of *LayerNorm*) into the parameters of the subsequent operator, explicitly removing these operations from the graph in inference mode.

Unlike operator fusion, which combines multiple operations into a single *kernel* during *codegen*, the *fold* structurally simplifies the graph before code generation. As a consequence, the *fold* mainly affects the fixed compilation cost, reducing *codegen* complexity, while its effects on per-execution latency tend to be smaller or non-deterministic.

For operations with fold, we assume $T_f(n) = a_f\, n + b_f$ to be the cost in backends that perform the *fold* operation before regular operator fusion. Thus, the equivalence point in the number of *batches* n for which $\Delta T = T_f(n) - T_b(n) = 0$, where $T_b(n)$ is the cost in the compiled backend without the fold operation, is given by:

$$n_{\text{eq}} = \text{MAX}\left(0,\ \frac{b_b - b_f}{a_f - a_b}\right), \qquad \text{provided } \Delta a \neq 0.$$

(6) `JSON` *construction.* For each *(model, input, backend)*, including the models with `fold`, we generate a `JSON` that consolidates the benchmark state. It includes the experiment identification, time metrics, fused *kernels*, and other metadata.

3.4 Execution per Compiler and Model

For each (**compiler**, **model**) pair, we run the pipeline [Section 3.3]: (i) graph compilation, (ii) input preparation, (iii) execution of the chosen protocol (10 *warmups* and $n = 50$), (iv) computation of mean, standard deviation, and CI (95%), and (v) export of metrics and artifacts.

3.5 Artifact Analysis Script (TVMScript, HLO, Triton)

We implemented a **Python script** that performs textual *parsing* of the program tree and produces an artifact.

Reproducibility and Code Availability

The source code, measurement scripts, and artifacts (HLO/TVM-Script/Triton) are available at: https://github.com/kameczera/compilers-benchmark

4 Evaluation

The metrics collected include: (i) the number of *kernel* launches at the start and end of the pipeline; (ii) the number of internal and external *kernels*; and (iii) specific decompositions (Triton *kernels*, for example). For **TVM** we followed the pipeline indicated in the documentation. For **TorchInductor**, we followed direct execution via PyTorch. For **XLA**, we report the number of fusions and custom calls.

The PyTorch and XLA models were loaded from native libraries. For TVM, an export step from the PyTorch models was required. This step affects the TVM metrics, since the export process tends to fragment *kernels*. Some operations that are treated as atomic in PyTorch (or encapsulated in optimized library calls) are decomposed by TVM into multiple simpler operators in its IR, which increases the number of generated *kernels* and reduces subsequent fusion opportunities.

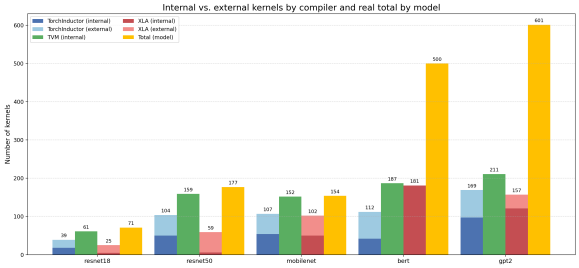

Figure 1: Fusion rate per model, considering only internal *kernels*.

In Figure 1 we see the rate of fused kernels across the three backends for the four models. Although both **TorchInductor** and **XLA** are able to fuse more *kernels* than **TVM**, the fusion rate of **XLA** is higher across all models except BERT (Figure 1). The dominance of **TorchInductor** and **XLA** is clearer in Figure 3. There, we notice a greater weight of external dependencies (external *kernels*), which also affects operator fusion time. In Figure 3 we also observe the number of internal *kernels* per backend in each model.

We caution that these fusion rates are only partially comparable across backends, because the three compilers do not ingest the same intermediate representation of a given model. As noted above, the PyTorch→TVM export fragments operators that PyTorch treats as atomic, inflating TVM's kernel count and lowering its apparent fusion rate. A strictly isonomic comparison would require feeding all backends the same operator-level representation and applying identical pattern-rewriting rules, so that the same fusion effects could be triggered everywhere; we therefore read these rates as backend-internal indicators rather than as a head-to-head ranking, and leave such a normalized comparison as future work.

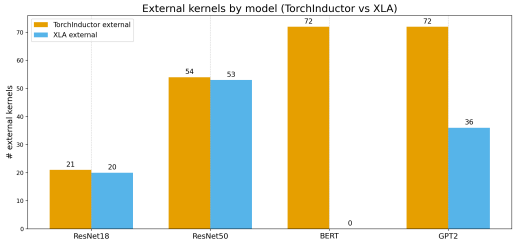

Figure 2: Number of external *kernels*.

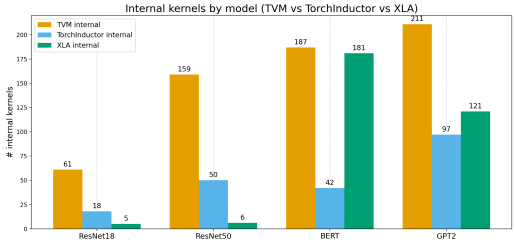

Figure 3: Number of internal *kernels*.

Table 1: ResNet-18 and ResNet-50 — compilation (ms) and execution time (mean ± sd, ms; $n = 50$, 10 warmups). Best per metric in blue. 95% CIs are derivable from sd and n.

		ResNet-18		ResNet-50	
Framework	**Input (N,C,H,W)**	**Comp.**	**Exec.**	**Comp.**	**Exec.**
TorchInductor	(1, 3, 224, 224)	8550	1.76 ± 0.03	13902	3.55 ± 0.04
	(16, 3, 224, 224)	6233	11.78 ± 0.18	13125	29.20 ± 0.35
	(64, 3, 224, 224)	6348	40.51 ± 0.42	11952	107.14 ± 1.29
	(1, 3, 512, 512)	6168	4.70 ± 0.05	12373	10.73 ± 0.13
	(1, 3, 1024, 1024)	5929	15.07 ± 0.16	11767	37.16 ± 0.45
TVM	(1, 3, 224, 224)	10273	6.94 ± 0.07	10273	19.47 ± 0.18
	(16, 3, 224, 224)	10004	82.53 ± 0.62	10004	247.63 ± 2.23
	(64, 3, 224, 224)	9899	321.09 ± 1.90	9896	986.00 ± 8.87
	(1, 3, 512, 512)	4992	27.00 ± 0.31	9648	91.01 ± 0.82
	(1, 3, 1024, 1024)	9256	103.97 ± 0.84	9256	347.04 ± 3.12
XLA	(1, 3, 224, 224)	2457	1.99 ± 0.03	4748	4.21 ± 0.04
	(16, 3, 224, 224)	2917	11.39 ± 0.15	5936	29.71 ± 0.30
	(64, 3, 224, 224)	4640	38.79 ± 0.39	9581	101.00 ± 1.01
	(1, 3, 512, 512)	3050	4.17 ± 0.05	4671	10.70 ± 0.11
	(1, 3, 1024, 1024)	3297	12.75 ± 0.12	6028	33.77 ± 0.34

4.1 Residual Networks

In the **ResNet-18** and **ResNet-50** architectures, the three backends reveal structural differences in how each system handles fusions, granularity, and delegation of operations to external libraries.

TVM exhibits the largest number of *kernels* due to its form of representation: the *convolution* and *batch normalization* operations arrive marked as *opaque*, and the compiler has no visibility to apply aggressive fusions. Even the *ReLU* remains isolated. The result is a fragmented pipeline, with simple fusions and little structural reduction of the graph.

TorchInductor relies heavily on **cuDNN** for convolutions, which prevents it from fusing convolution, *BatchNorm*, and activation into a single *kernel*. However, it performs *BatchNorm+ReLU* fusions within Triton *kernels*. There are fewer launches and the graphs are more compact than TVM's. Nevertheless, the use of Triton for elementwise operations and reductions increases code-generation time, **which grows proportionally with the model** (Table 1).

This reduces the total number of launches and, in general, results in more compact graphs than TVM's. However, this hybrid approach carries a cost: using Triton for numerous *elementwise* operations and reductions increases *codegen* time and causes compilation time to grow proportionally with model size, especially noticeable when moving from ResNet-18 to ResNet-50.

For XLA, based on the analysis of the HLOs, we observe that the compiler performs aggressive fusions, combining *convolution + bias + batch norm + activation* into a single cuDNN call. This fusion reduces the number of final *kernels* and results in more cohesive pipelines. Experimentally, this behavior showed a direct impact on reducing compilation time compared to the other backends.

Table 2: BERT — compilation (ms) and execution time (mean ± sd, ms) per input (*batch, seq_len*); $n = 50$, 10 warmups. Best per metric in blue.

Framework	Input	Comp.	Exec.
TorchInductor	(1, 64)	13449	3.49 ± 0.63
	(1, 128)	12071	5.11 ± 0.73
	(8, 128)	11396	31.51 ± 0.95
TVM	(1, 64)	2492	6.78 ± 0.34
	(1, 128)	1564	9.85 ± 0.49
	(8, 128)	1464	63.00 ± 1.26
XLA	(1, 64)	5474	3.88 ± 0.26
	(1, 128)	5074	5.27 ± 0.25
	(8, 128)	5317	29.95 ± 0.32

4.2 Attention-Based Architectures (BERT)

The **BERT** model has a distinct fusion pattern that reflects its architecture centered on *matmuls* [10], QKV projections, and normalization and activation operations.

TVM tends to produce smaller and more numerous *kernels*. This leads to graph fragmentation, similar to what was seen in the ResNets, but in BERT its fusion rate is higher thanks to the structure, which has long elementary chains that are easily collapsible.

TorchInductor explores fusions aimed at minimizing memory access. It delegates the *matmuls* to external libraries (for example, cuBLAS) [1] and focuses on the Triton *kernels*. From the direct analysis of the artifacts and kernels generated by TorchInductor, we observed that the compiler synthesizes structures that correspond, in practice, to typical *Transformer* blocks. These kernel templates are reused throughout the network, which leads to a reduction in the number of launches, although at the cost of higher compilation time.

XLA ends up with more *kernels* than TorchInductor, but practically all of them are **HLO fusions**: from *GEMM* fusions (`gemm_fusion_dot`, with concatenated QKV weights and embedded epilogues) to fusions dedicated to attention `softmax` (`triton_softmax_computation`), and large `kLoop`/`kInput` fusions that implement full `LayerNorm` and `GELU`. These *GEMM* fusions correspond to **the same 72 matrix multiplications observed in TorchInductor**;

Table 3: GPT-2 — compilation (ms) and execution time (mean ± sd, ms) per input (*batch, seq_len*); $n = 50$, 10 warmups. Best per metric in blue.

Framework	Input	Comp.	Exec.
TorchInductor	(1, 16)	15003	3.09 ± 0.33
	(1, 128)	13327	6.12 ± 0.60
	(8, 128)	19121	30.40 ± 0.75
	(8, 256)	18353	58.56 ± 0.74
TVM	(1, 16)	8253	7.80 ± 0.39
	(1, 128)	10312	10.55 ± 0.53
	(8, 128)	9776	41.21 ± 1.24
	(8, 256)	9549	79.11 ± 1.98
XLA	(1, 16)	2797	2.81 ± 0.17
	(1, 128)	3892	5.11 ± 0.28
	(8, 128)	5784	30.84 ± 0.72
	(8, 256)	5562	58.38 ± 0.54

however, being highly specialized internal fusions, they already incorporate *bias* operations, QKV concatenation, and *layout* adjustments within a single *kernel*. When the *batch* increases, the cost of these *GEMMs* **comes to dominate the total execution time**, which renders the overhead due to the larger number of *kernels* irrelevant. This behavior is especially visible in the (8, 128) scenario (Table 2). We stress that we only claim one backend to be faster than another when their 95% confidence intervals do not overlap. At (1, 128), for instance, TorchInductor (5.11 ± 0.73 ms) and XLA (5.27 ± 0.25 ms) have overlapping intervals and are therefore statistically indistinguishable, so we refrain from ranking them at that input; the meaningful gaps appear at larger batches such as (8, 128), where the GEMM cost dominates and the intervals separate.

In short, the BERT model highlights how architectural differences affect each compiler's strategy.

4.3 Autoregressive Attention-Based Models (GPT-2)

The GPT-2 blocks follow a relatively linear decoder-only pattern—autoregressive attention and MLP, each preceded by LayerNorm and followed by GELU activation, repeated across all layers [11].

TVM cannot incorporate long epilogues, such as *softmax* or *LayerNorm*, into larger *kernels*, since many are marked as opaque in the IR. Even so, TVM fuses short epilogues, such as *bias* addition and, in some cases, *GELU*, in addition to consolidating elementary chains and nearby normalizations.

TorchInductor adopts the same strategy: it delegates the *kernels* to external libraries and synthesizes Triton *kernels* combining normalization, activations, and *softmax*. The Inductor has an "expanded epilogue" pattern that reduces the number of *kernels*, with fusions involving **LayerNorm**, **GELU**, and **softmax**. As with BERT, however, this strategy implies a higher compilation cost, given the size and complexity of the Triton *kernels*.

XLA presents a balanced strategy between granularity and integration with external libraries. XLA has the ability to consolidate multiple auxiliary operations around the *matmuls*, resulting in larger *kernels*, which leads it to obtain **faster execution times in most configurations** (Table 3).

Although the final fusion rates between **TorchInductor** and **XLA** are close, XLA's more structured fusion pattern allows it to maintain superior execution-time performance on GPT-2.

In terms of execution time, **XLA** was the best across **all** models, showing that both fusion granularity and fusion quality are relevant metrics and must be analyzed together.

4.4 Fold

Table 4: ResNet-18 (TorchInductor) — effect of folding BN into Conv: compilation/execution times (ms).

Input	Comp. w/o fold	Comp. w/ fold	Δ Comp.	Exec. w/o fold	Exec. w/ fold	Δ Exec.
(1,3,224,224)	8550	4305.06	−49.6%	1.76	1.97	+12.0%
(16,3,224,224)	6233	2566.06	−58.9%	11.78	11.70	−0.7%
(64,3,224,224)	6348	2790.42	−56.0%	40.51	42.69	+5.4%
(1,3,512,512)	6168	2909.80	−52.9%	4.70	4.80	+2.2%
(1,3,1024,1024)	5929	2582.70	−56.5%	15.07	15.05	−0.1%

Table 5: ResNet-50 (TorchInductor) — effect of folding BN into Conv: compilation/execution times (ms).

Input	Comp. w/o fold	Comp. w/ fold	Δ Comp.	Exec. w/o fold	Exec. w/ fold	Δ Exec.
(1,3,224,224)	13902	11467.60	−17.5%	3.55	3.48	−2.0%
(16,3,224,224)	13125	7401.21	−43.6%	29.20	28.832	−1.3%
(64,3,224,224)	11952	8576.11	−28.3%	107.14	105.917	−1.1%
(1,3,512,512)	12373	7753.59	−37.3%	10.73	10.601	−1.2%
(1,3,1024,1024)	11767	7618.43	−35.3%	37.16	37.088	−0.2%

Fold (ResNets). Applying the *fold* before the fusion pass reduces compilation time in both ResNets (Tables 4 and 5). The impact varies with model size. The **BatchNorm** *fold* acts as a simple structural optimization inside the *kernels*, facilitating the writing of their fusion. However, it significantly changes execution-time performance, as in the first row of Table 4 (+12% in ResNet-18).

Table 6: BERT (TorchInductor) — effect of folding the affine part of LayerNorm into Linear: compilation/execution times (ms).

Input	Comp. w/o fold	Comp. w/ fold	Δ Comp.	Exec. w/o fold	Exec. w/ fold	Δ Exec.
(1,64)	13449	6345.70	−52.8%	3.49	3.22	−7.7%
(1,128)	12071	6106.58	−49.4%	5.11	5.25	+2.8%
(8,128)	11396	6119.32	−46.3%	31.51	30.28	−3.9%

Fold (BERT).. In BERT, the **LayerNorm** (LN) at inference time splits into two parts: (i) the *normalization* and (ii) the *affine part* (γ, β) (see Appendix A). In **TorchInductor**, when LN and its affine part appear composed in the graph, the compiler generates a single Triton *kernel* that includes normalization, element-wise operations (`mul`/`add`), and multiple additional *loads*. This combination makes the *kernel* denser and raises the *codegen* cost.

By applying the **fold**—absorbing γ and β into the subsequent `Linear` (scaling the columns of W and adjusting the bias)—the LN becomes merely a normalization *without affine*. This removes the extra operations from the Triton *kernel* and significantly reduces its density, simplifying the code generator's work and thus **reducing**

compilation time, even though the number of *kernels* remains the same.

The *fold* also reduces the gap relative to **XLA**. In **XLA**, the LN appears inside larger fusion regions, compiled monolithically, without Python/Triton orchestration and without materializing the affine step as an isolated operation.

In summary: **TVM** exhibits higher granularity (more *kernels*) and more conservative fusions; **TorchInductor** combines Triton *kernels* with external library calls; **XLA** performs more aggressive fusions on Transformers but retains custom calls.

TorchInductor tends to generate separate *kernels* for the convolution (delegated to libraries) and for subsequent operations compiled via Triton; **XLA**, on the other hand, usually fuses these operations into a single graph and delegates execution, reducing code generation and launches.

The tests with the *fold* before graph conversion **show a relevant improvement in PyTorch's compilation time**. This indicates an inefficient optimization order in the PyTorch backend. Our work argues that the *fold* operation should be performed before operator fusion, since it simplifies the graph before *codegen*. Figure 4 supports our claim.

Execution-time gains are not deterministic in any of the three models (ResNets and BERT), due to the nature of code optimization. The analysis shows that **a longer compilation time does not, by itself, imply better fusions**. In ResNet-50, we observed an execution-time improvement after the ***fold*** in all test cases. We therefore see the need to build a benchmark and an algebraic model to decide which ML backend to use in terms of the relationship between compilation time and execution time. In Figure 4, for example, we compute the equivalence number n_{eq} (defined in Section 3) for the *fold* fusion. Negative numbers were simplified to zero, since they mean that 0 *batches* are required to obtain an execution-time gain.

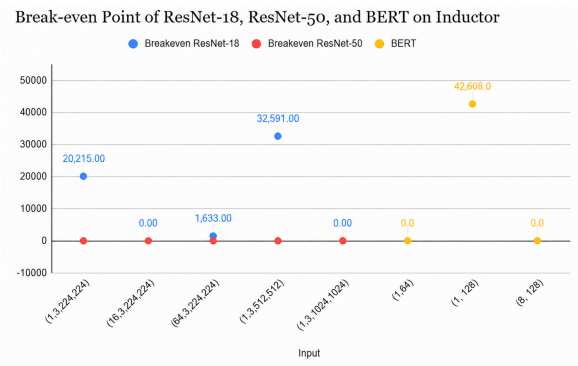

Figure 4: Equivalence point (number of executions from which it is worth compiling with the fold pass) on the TorchInductor backend.

This number is important because it is the break-even point between the cost of an optimization and the benefit it yields at execution. In ML systems, operator fusion is expensive: if a model is executed only a limited number of times, this cost may not be offset by the reduction in per-execution latency, and may even outweigh it. Making this threshold explicit is therefore useful whenever a system must decide, at deployment time, whether a given fusion or *fold* pass is worth applying for the expected number of executions.

We performed the analysis of the fusion pass in isolation, but the proposed benchmark can be extended to other passes and other models.

4.5 Proposed Benchmark

We developed a *benchmark* in which the user chooses the **model input** ($N{\times}C{\times}H{\times}W$) and the system runs, for each compiler, the standardized measurement of **compilation time** and **execution time per inference** for that input. At the end, the *benchmark* reports (i) *which compiler provides the best gain* for the number of executions of interest, and (ii) the **ranges of executions** in which *each* compiler excels.

To ensure fairness in the comparison, **all compilers were configured equivalently**: the same architecture (ResNet-18/50), the same precision, the same device, the same layout when applicable (e.g., *channels-last*/NHWC), `eval()` enabled, identical *warmup* windows and number of iterations, plus congruent execution parameters (e.g., autotuning enabled when available). The orchestrator unifies metric collection and JSON serialization, ensuring reproducibility.

We model the total cost of each *backend* b as $T_b(n) = a_b\, n + b_b$, where a_b is the **average time per inference** and b_b is the **compilation time**. By constructing the **lower envelope** of these lines, we obtain the **crossover points** and, consequently, the **dominance ranges** of each compiler.

Report (`JSON`) and quick reading. The *benchmark* generates a `JSON` file with: **meta** (experiment context), **raw** (measurements per *backend*: `compile_ms`, `exec_ms`, `ttfb_ms`), and **recommendation** (the $T_b(n)$ model, crossover points, and dominance segments). The `best_at_runs_exec` field directly indicates the best *backend* for the given n.

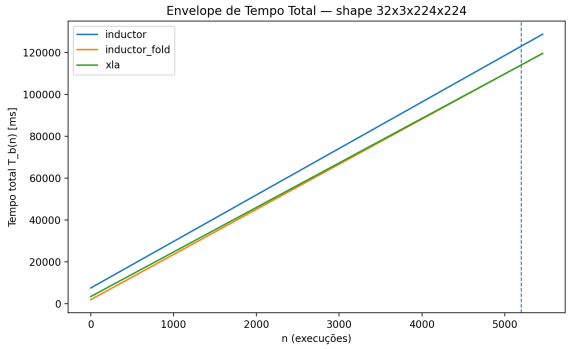

Figure 5: Example benchmark plot: lines $T_b(n) = a_b n + b_b$ per *backend*, with the lower envelope (dominance regions) and crossover points. In this example, `inductor_fold` was best up to execution 5,197; from there on, XLA dominated.

How to read Figure 5. The y-axis starts at `compile_ms` (intercept) and the slope is `exec_ms`; the lower envelope highlights the winner

in each range of n; the intersections correspond to the crossover points (dashed vertical lines). If there are no intersections, the JSON comes with an empty `breakpoints` field and a single `segment`.

5 Related Work

Operator fusion can be seen as the computation-graph analogue of classical *peephole optimization*. Fraser's machine-independent peephole optimizer characterized valid local rewrites over a sequence of machine instructions by describing each instruction's effect as a Register Transfer List, replacing a window of instructions by a single equivalent one whenever such a description existed [6]. Operator fusion plays the same role one level up: where peephole optimization rewrites a window of three-address-code instructions into an equivalent instruction, fusion rewrites a window of computation-graph operators into an equivalent operator that preserves the observable semantics. In this sense, fusion is to the operators of a computational graph what peephole optimization is to instructions in a three-address-code representation. Among graph-level approaches, *Optimus* minimizes memory accesses over the computation DAG through an offline dynamic-programming algorithm and an online buffer-budget variant [2]. A practical limitation of such optimal-search formulations, however, is convergence: under a fixed time budget they often fail to reach the optimal fusion plan, and Canesche et al. [3] report that exhaustive search methods of this kind fail to converge on almost every large benchmark in their evaluation, which motivates the faster, heuristic strategies adopted by production compilers such as XLA, TVM, and TorchInductor.

6 Conclusion

This work presented a unified protocol for evaluating machine learning compilers (TorchInductor, XLA, and TVM), controlling *inputs*, warmup, and the number of iterations, and introduced a decision criterion based on the *lower envelope* of the total times $T_b(n) = a_b n + b_b$, where a_b captures execution latency and b_b the compilation cost. The comparative analysis showed that differences in the handling of *fusion* and of epilogues in libraries such as `cuDNN` explain much of the observed variation: XLA tends to benefit when it can delegate sequences such as `Conv+BatchNorm+ReLU` to a single *custom call*, whereas TorchInductor, on the current *inference* path, pays an overhead for *codegen* and additional launches when emitting `BN+ReLU` as a separate kernel (via Triton). We also observed that each backend's dominance depends on the n regime: in situations with few executions, choices with cheaper compilation win; as n dominates, strategies with a higher degree of kernel fusion prevail.

Limitations. Despite the methodological care, there is still room to broaden the statistical robustness (e.g., reporting confidence intervals for all scenarios) and to diversify the environment (other GPUs/CPUs and `driver`/`cuDNN` versions). In addition, aspects of the CFG export may affect the fusion of non-canonical kernels in some backends.

Another limitation lies in the linearity assumption of the model $T_b(n) = a_b n + b_b$ itself: by assuming that the per-batch cost is approximately constant and that all of the fixed cost is paid in the first batch, the model abstracts away cache effects, *autotuning* strategies, and possible recompilations triggered by dynamic *shapes*, which may make the effective behavior closer to a piecewise model than to a single line. In practice, the coefficients a_b and b_b should be read as summary statistics of the range of n that was measured, and not as exact predictions for any usage regime. Future work can refine this dimension by reporting goodness-of-fit metrics (for example, the regression's R^2) and by investigating nonlinear or *piecewise* models in scenarios with greater temporal variability.

Beyond the comparison between backends, this work also evaluated *fold* transformations in the **TorchInductor** graph, aiming to reduce the compilation cost without changing the model's semantics. In convolutional architectures (ResNet-18/50), folding `BatchNorm` into `Conv` reparameterizes the convolution's weights and biases and removes the explicit `BatchNorm` from the graph in *eval* mode. This does not turn the `Conv+BatchNorm+ReLU` sequence into a single kernel—unlike XLA, which often delegates the whole chain to a single *custom call*—but it reduces the volume of *codegen*, since reparameterizing the `Conv` is much cheaper than generating a separate Triton kernel for `BN+ReLU`.

In attention-based models, such as BERT, we apply an analogous *fold* on the `LayerNorm+Linear` combination: the affine part of `LayerNorm` (γ, β) is absorbed into the weights and biases of the subsequent `Linear`, so that TorchInductor now compiles only a "pure" `LayerNorm`, responsible solely for normalization. This eliminates the need for a more complex `LayerNorm` kernel (normalization + affine part) and reduces the associated *codegen*, similarly to the effect observed with `BatchNorm`.

A key observation of our experiments is that TorchInductor does not apply folding automatically on the inference path, yet by applying it manually we obtained consistent compilation-time reductions and, in several cases, execution-time speedups. Since the Inductor is already a heavily optimized backend, this points to a broader research direction: generalizing the fold transformation and implementing it as an automatic pass inside the Inductor. As a direct first step, we intend to enable the fusion of `Conv+BatchNorm+ReLU` on the *inference* path of **TorchInductor**. The proposal consists of (i) folding `BN` into `Conv` in `eval()` mode by adjusting (W', b'); (ii) triggering the `ReLU` epilogue directly in `cuDNN` via `aten::cudnn_convolution_relu(...)` (as happens in XLA); and (iii) integrating a *pattern-rewrite* (FX/`torch._inductor.pattern_matcher`) that detects `Conv`→`BN`→`ReLU` and falls back safely when the epilogue is not available. The evaluation of this fusion will be incorporated into our existing *benchmark*, adding the new point to the $T_b(n)$ *envelope*. We expect to reduce TTFB and latency through a smaller number of Triton kernels and launches, widening the ranges in which TorchInductor dominates—bringing it closer to the profile observed when XLA delegates epilogues to `cuDNN`.

Acknowledgments

The authors used **ChatGPT (OpenAI, 2025)** as a support tool during the preparation of this paper, restricted to the following purposes: refining the writing and checking textual coherence; reviewing the algebraic derivations of the kernel fusions in Appendix A; and assisting in writing and debugging portions of the benchmark code used in the experiments. All technical, methodological, and analytical content is the authors' own and was reviewed and validated

by the authors. This disclosure follows the SBLP generative-AI policy and the corresponding ACM, IEEE, and Springer guidelines.

A Derivation of the Analyzed Fusions

This appendix lists the three kernel fusions that are the focus of our benchmark. Throughout, $\odot$ denotes the element-wise (Hadamard) product, with broadcasting along the channel dimension when the operands' shapes differ.

BatchNorm + ReLU

A widely known fusion combines a batch normalization (`BatchNorm`) followed by a rectified linear unit (`ReLU`). Let $X \in \mathbb{R}^{N\times C\times H\times W}$ be the activation tensor (batch, channels, spatial dimensions); $\mu \in \mathbb{R}^C$ and $\sigma^2 \in \mathbb{R}^C$ the per-channel estimated mean and variance; $\gamma, \beta \in \mathbb{R}^C$ the learnable scale and shift; and $\varepsilon \in \mathbb{R}_{>0}$ the numerical-stability constant. `BatchNorm` computes

$$\text{(I)} \qquad Y = \frac{X-\mu}{\sqrt{\sigma^2+\varepsilon}} \odot \gamma + \beta. \tag{1}$$

The `ReLU` is then applied element-wise to the output:

$$\text{(II)} \qquad Z = \max(0, Y). \tag{2}$$

Realizing these two operations requires two kernels, (I) and (II). However, both can be carried out in a single kernel:

$$\text{(III)} \qquad y = \max\left(0,\ \gamma \cdot \frac{x-\mu}{\sqrt{\sigma^2+\varepsilon}} + \beta\right). \tag{3}$$

Modern deep-learning compilers nonetheless cannot fuse every `BatchNorm+ReLU` pair, for reasons of coercion, dynamic behavior, and structural constraints in the program CFG (Section 2).

Conv + BatchNorm + ReLU

In computer vision, the `Conv--BatchNorm--ReLU` chain is a natural fusion candidate. At inference, `BatchNorm` is a fixed linear transformation and can be absorbed into the convolution's weights and bias $(\tilde{W}, \tilde{b})$; the `ReLU`, being element-wise, enters as an epilogue of the same kernel. The compiler then (i) recomputes $(\tilde{W}, \tilde{b})$ incorporating the BN and (ii) applies the `ReLU` at the end of the kernel, eliminating intermediate tensors and extra launches. The arithmetic cost of the convolution is unchanged; the gain comes from reduced data movement and kernel orchestration.

Formally, let $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$ be the input and $W \in \mathbb{R}^{C_{out}\times C_{in}\times K_H\times K_W}$, $b \in \mathbb{R}^{C_{out}}$ the convolution parameters. The pre-activation output is

$$\text{(IV)} \qquad U_{n,c,h,w} = \sum_{c'=1}^{C_{in}} \sum_{r,s} W_{c,c',r,s}\, X_{n,c',h+r,w+s} + b_c. \tag{4}$$

`BatchNorm` at inference, with parameters $\mu, \sigma^2, \gamma, \beta \in \mathbb{R}^{C_{out}}$ and $\varepsilon > 0$, is

$$\text{(V)} \qquad Y_{n,c,h,w} = \frac{U_{n,c,h,w}-\mu_c}{\sqrt{\sigma_c^2+\varepsilon}}\,\gamma_c + \beta_c. \tag{5}$$

The `ReLU` (II) is then applied element-wise. Rewriting $Y_{n,c,h,w}$ in the form of kernel (III), we absorb the `BatchNorm` into the convolution parameters by defining

$$\text{(VI)} \quad \tilde{W}_{c,c',r,s} = \frac{\gamma_c}{\sqrt{\sigma_c^2+\varepsilon}}\, W_{c,c',r,s}, \qquad \text{(VII)} \quad \tilde{b}_c = \frac{\gamma_c}{\sqrt{\sigma_c^2+\varepsilon}}\,(b_c-\mu_c) + \beta_c. \tag{6}$$

Substituting into Y and applying the `ReLU`, we obtain the `Conv+BatchNorm+ReLU` fusion in a single kernel:

$$\text{(VIII)} \qquad Z_{n,c,h,w} = \max\left(0,\ \sum_{c'=1}^{C_{in}} \sum_{r,s} \tilde{W}_{c,c',r,s}\, X_{n,c',h+r,w+s} + \tilde{b}_c\right). \tag{7}$$

The inner term is exactly of the form of kernel (III), now with the convolution already reparameterized by $(\tilde{W}, \tilde{b})$.

LayerNorm (affine part) + Linear

Analogously to the `Conv--BatchNorm--ReLU` case, we can exploit the structure of `LayerNorm` (LN) in `LN--Linear` chains, common in Transformer models such as BERT. For a single activation vector $h \in \mathbb{R}^d$, decompose the inference-time LN into (i) normalization (centering and rescaling by the standard deviation) and (ii) the affine part (scaling by γ and shifting by β). Let

$$\text{(IX)} \qquad \hat{h} = \mathrm{Norm}(h) \tag{8}$$

be the purely normalizing part of the LN, i.e., the vector centered and normalized using the mean and variance of h. The full LN is

$$\text{(X)} \qquad \tilde{h} = \gamma \odot \hat{h} + \beta, \tag{9}$$

where $\gamma, \beta \in \mathbb{R}^d$ are the learnable scale and shift parameters. Suppose a `Linear` layer with weights $W \in \mathbb{R}^{m\times d}$ and bias $b \in \mathbb{R}^m$ follows. Its output is

$$\text{(XI)} \qquad y = W\tilde{h} + b = W\big(\gamma \odot \hat{h} + \beta\big) + b. \tag{10}$$

Expanding,

$$\text{(XII)} \qquad y = \underbrace{W\,\mathrm{diag}(\gamma)}_{W'}\,\hat{h} + \underbrace{(W\beta + b)}_{b'}. \tag{11}$$

Defining

$$\text{(XIII)} \qquad W' = W\,\mathrm{diag}(\gamma), \qquad b' = W\beta + b, \tag{12}$$

we rewrite the `LN+Linear` composition as

$$\text{(XIV)} \qquad y = W'\,\mathrm{Norm}(h) + b'. \tag{13}$$

That is, the affine part of the `LayerNorm` (γ, β) can be absorbed beforehand into the `Linear` parameters, leaving the LN as a purely normalizing operation (without affine). In implementation terms, this lets the compiler (i) treat the LN as a simpler normalization kernel (fewer `mul`/`add` operations and fewer parameter tensors) and (ii) keep the `Linear` as an unchanged GEMM kernel, only with reparameterized weights and bias (W', b'). As in the `Conv--BatchNorm--ReLU` case, the total arithmetic cost of the `Linear` does not change; the benefit comes from simplifying the normalization kernels and reducing codegen work, especially in models with multiple LNs per block/layer.

REFERENCES

[1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2022. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *ACM* (2022), 929–947. doi:10.1145/3620665.3640366

[2] Xuyi Cai, Ying Wang, and Lei Zhang. 2022. Optimus: An Operator Fusion Framework for Deep Neural Networks. *ACM Transactions on Embedded Computing Systems* 22, 1 (2022), 1–26. doi:10.1145/3520142

[3] Michael Canesche, Vanderson Martins do Rosario, Edson Borin, and Fernando Magno Quintão Pereira. 2025. Fusion of Operators of Computational Graphs via Greedy Clustering: The XNNC Experience. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction (CC '25)*. ACM, Las Vegas, NV, USA, 117–127. doi:10.1145/3708493.3712689

[4] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.

[5] Alain Darte. 1999. On the complexity of loop fusion. In *Proceedings of the 1999 International Conference on Parallel Architectures and Compilation Techniques*. IEEE, 149–157. Cat. No. PR00425.

[6] Christopher W. Fraser. 1979. A Compact, Machine-Independent Peephole Optimizer. In *Conference Record of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '79)*. ACM, San Antonio, Texas, USA, 1–6. doi:10.1145/567752.567753

[7] Ken Kennedy and Kathryn S. McKinley. 1993. Maximizing Loop Parallelism and Improving Data Locality via Loop Fusion and Distribution. In *Proceedings of the 6th International Workshop on Languages and Compilers for Parallel Computing*. Springer-Verlag, Berlin, Heidelberg, 301–320.

[8] Chris Leary and Todd Wang. 2017. XLA: TensorFlow, Compiled. In *TensorFlow Dev Summit*. https://www.tensorflow.org/xla

[9] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion. *ACM* (2021), 883–898. doi:10.1145/arXiv:2108.13342

[10] Suchita Pati, Shaizeen Aga, Nuwan Jayasena, and Matthew D. Sinclair. 2021. Demystifying BERT: Implications for Accelerator Design. In *IEEE International Symposium on Workload Characterization (IISWC 2021)*. IEEE, 109–120. doi:10.1109/IISWC53511.2021.00019

[11] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report* (2019).

[12] Daniel Snider and Ruofan Liang. 2023. Operator Fusion in XLA: Analysis and Evaluation. https://arxiv.org/abs/2301.13062. arXiv:2301.13062.

[13] Michael E. Wolf and Monica S. Lam. 1991. A Data Locality Optimizing Algorithm. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 30–44.